Putting the loop to work

Week 10, Lecture 1 · Browser Use: How LLM Agents Drive the Web

Summer 2026

What we will cover

- Scoping a real task: four tests before you write any code
- Choosing built-in vs custom tools: the decision criteria
- Designing the output schema: from prose to a typed Pydantic model
- Auth and secrets in production: profiles, placeholders, and domain restrictions
- The synthesis view: how all ten weeks fit into one running agent

Part 1: scoping a real task

Four tests before writing code

- **Stopping condition**: can the agent tell when it is done? Name the condition in the task string.
- **DOM-visible steps**: does every action target an element the selector map can index?
- **Auth external**: login handled before the task runs, not inside it.
- **One job**: no "and then" linking two independent goals. Split tasks that branch.

What makes a task fail before it starts

- Vague stopping condition: "find something interesting on this page"
- Steps that require non-DOM content: PDF plugins, canvas elements, closed shadow DOM
- Credentials in the task string (they appear in the LLM context and in logs)
- Two tasks welded together: "log in, then extract, then post the result to a webhook"
- The agent loop cannot fix a bad task string. Fix it first.

Writing a task string that works

```
BAD:  "Monitor the product page and tell me if anything changes."

GOOD: "Go to the product page at x-product-url.
       Extract the product name, current price in USD, and whether
       it is in stock. If price requires choosing a variant,
       select the first available variant.
       Return the result. If the price is not visible after two
       navigation attempts, set price_usd to -1."
```

- Stopping condition: explicit
- Variant handling: explicit escape named
- URL: passed via sensitive_data, not hardcoded in the string

Part 2: built-in vs custom tools

The built-in action vocabulary

Ten actions ship with browser-use (browser-use contributors, 2026, service.py):

- **Navigation**: navigate to URL, go back, open tab, switch tab
- **Interaction**: click element, type text, scroll
- **Extraction**: extract page content
- **Control**: done

These cover most navigational and data-extraction tasks without any additional code.

When to write a custom action

Write a `@tools.action` handler when a step requires:

- Calling an external API or webhook mid-task
- Reading or writing a local file
- Complex extraction logic (table parsing, multi-source joins)
- Domain restriction: the action should only run on specific sites
- A side effect the built-in actions cannot produce

If built-in `extract page content` returns what you need, do not write a custom extractor (browser-use team, 2026, custom-functions docs).

Custom action anatomy

```python
from browser_use import Tools
from browser_use.agent.views import ActionResult

tools = Tools()

@tools.action(
    "Save the extracted price to a local file for the price-monitor dashboard.",
    domains=["shop.example.com"],
)
async def save_price(product_name: str, price_usd: float) -> ActionResult:
    with open("/tmp/prices.jsonl", "a") as f:
        f.write(f'{{"name": "{product_name}", "price": {price_usd}}}\n')
    return ActionResult(extracted_content="Price saved.")
```

- Typed parameters: the LLM sees the schema and knows how to call it
- `domains=`: restricts the action to `shop.example.com` only
- Returns `ActionResult`, not raw data

Part 3: designing the output schema

From prose to typed output

Default: the agent returns a prose string when it calls `done`.

Problem: prose varies across runs, cannot be type-checked, breaks callers when format shifts.

Solution: pass an `output_model` Pydantic class. The agent is instructed to return JSON matching that schema (browser-use team, 2026, output-format docs).

Schema design rules

```python
from pydantic import BaseModel, Field
from typing import Optional

class ProductResult(BaseModel):
    name: str = Field(description="Product name as shown on the page")
    price_usd: float = Field(description="Price in USD, no symbol")
    in_stock: bool = Field(description="True if add-to-cart is available")
    sku: Optional[str] = Field(None, description="SKU if shown, else None")
```

- Start from what the **caller** needs, not what the page shows
- Flat models: one level deep, no nested objects unless the data demands it
- `Optional` only when the page may genuinely not have that data
- `Field(description=...)` gives the LLM context for each slot

Part 4: auth and secrets in production

Two distinct problems

Where credentials come from: `sensitive_data` maps placeholder names to real values. The LLM sees only the placeholder. The framework substitutes the real value at runtime (browser-use team, 2026, sensitive-data docs).

How browser state persists: `user_data_dir` on the `BrowserSession` saves cookies, local storage, and session tokens to disk. A second agent reuses that state without logging in again.

The two-step auth pattern

```
# Step 1: login agent (run once, or when the session expires)
login_agent = Agent(
    task="Log in with x-username and x-password.",
    llm=llm,
    browser_session=BrowserSession(user_data_dir="/tmp/profile"),
    sensitive_data={"x-username": "alice@ex.com", "x-password": "s3cr3t"},
)
await login_agent.run()

# Step 2: task agents (reuse the saved session)
task_agent = Agent(
    task="Extract the account balance from the dashboard.",
    llm=llm,
    browser_session=BrowserSession(user_data_dir="/tmp/profile"),
    output_model=BalanceResult,
)
result = await task_agent.run()
```

Restricting blast radius

- `allowed_domains` on the agent: restricts all navigation to listed domains
- `domains=` on a `@tools.action`: restricts one custom action to specific sites
- Both reduce the damage if page content attempts a prompt-injection attack (week 9)
- The combination of `sensitive_data` + `allowed_domains` is the production-safe pattern

Part 5: the synthesis view

All ten weeks in one agent

Week	Concept	Where it appears
1	perceive-decide-act	the run loop in service.py
2	selector map	element indices the LLM clicks
3	BrowserSession	session config, user_data_dir
4	AgentOutput	structured decision at each step
5	tools registry	action dispatch
6	custom action, output_model	@tools.action, ProductResult
7	browser profile, auth	user_data_dir, sensitive_data
8	reliability	max_steps, max_failures, task framing
9	eval, deployment	task-set scorecard, CLI or cloud

> The goal is to make the action space as large as possible while keeping the loop as small as possible. Abstractions compound errors. A minimal loop with a well-defined action space compounds capability instead.

Take-aways

1. Scope the task first. A vague task string is the most common cause of agent failure; no configuration parameter fixes it.
2. Use built-in actions until a step requires an external system, a file, or domain restriction. Only then write a custom `@tools.action`.
3. Design the output schema from the caller's perspective. Flat Pydantic models with `Field(description=...)` annotations outperform ad-hoc prose extraction.
4. Keep secrets out of the LLM context with `sensitive_data`. Persist sessions with `user_data_dir`. Restrict with `allowed_domains`.

What’s next

- **Reading:** Week 10 reading ties all ten weeks into a single worked example and introduces the task-set scorecard
- **Lecture 2 (this week):** a complete agent traced end to end, reliability guards, measuring success, and presenting the design
- **Section (this week):** Capstone studio: scope, build, test against a task set, and defend your design
- **Capstone project:** due this week at /c/browser-use-26su/hw/capstone